

GearUp — Project Walkthrough & Examiner Tour

GearUp is a sports & outdoor gear rental marketplace. Three roles use one app; the UI and route access adapt to whoever is signed in.

- Live frontend: <https://gear-up-frontend-hasib.vercel.app>
- Live backend API: <https://gearup-rental-api.vercel.app>
- Backend repo: <https://github.com/Sheikhasib/GearUp-Rent-Sports-Outdoor-Gear-Instantly--Backend-API->
- Frontend repo: <https://github.com/Sheikhasib/GearUp-Frontend>

This document is in two parts: a **complete reference tour** (understand every feature and where it lives in the code) and a **live demo script** (what to click and say when walking an examiner through the deployed site).

Part 1 — Complete reference tour

1. Tech stack & architecture

Layer	Technology
Framework	Next.js 16 (App Router, Turbopack, TypeScript)
Styling	Tailwind CSS v4, shadcn/ui, Radix UI, <code>next-themes</code> (dark mode)
Client state	React Query + Zustand
Auth	Cookie-based JWT; middleware (<code>proxy.ts</code>) verifies with <code>jsonwebtoken</code>
Images	<code>next-cloudinary</code> upload widget → <code>res.cloudinary.com</code> (only whitelisted host in <code>next.config.ts</code>)
Payments	SSLCommerz sandbox redirect
Backend	Express + Prisma + PostgreSQL (deployed on Vercel)

Two-server split (Server Actions / Client components):

- `process.env.BACKEND_API_URL` — server components, server actions, middleware.
- `process.env.NEXT_PUBLIC_BACKEND_API_URL` — client components (via `lib/api/client.ts`).

2. Roles & permissions

Role	Access	Key abilities
CUSTOMER	<code>/customer-dashboard/**</code>	Browse/filter gear, place rentals, pay via SSLCommerz, track & cancel orders (while PLACED), review after return
PROVIDER	<code>/provider-dashboard/**</code>	Add/edit/delete own gear (Cloudinary upload), manage incoming orders through status lifecycle
ADMIN	<code>/admin-dashboard/**</code>	Platform stats, suspend/activate users, moderate all gear & orders, manage categories

Anyone may browse public pages. Register at `/register` with role selection; login at `/login` . Seeded admin: `admin@gearup.com` / `Admin123!` .

3. Auth flow & route guarding

`proxy.ts` middleware runs on every route except static assets.

1. Reads `accessToken` / `refreshToken` `httpOnly` cookies.
2. Verifies access token with `JWT_ACCESS_SECRET` . If expired but refresh is valid, calls `POST /api/auth/refresh-token` and re-sets the access cookie.
3. Guards:
 - Logged-in users hitting `/login` / `/register` → redirected to their role dashboard.
 - Unauthenticated users on non-public routes → `/login?redirectTo=...` .
 - Role mismatch (e.g. CUSTOMER opening `/admin-dashboard`) → `/not-found` .
4. `/payment/success` and `/payment/cancel` verify real payment status server-side before rendering.

Auth endpoints live in `app/(authGroup)/_actions/authActions.ts` ; current user fetched via `service/getMe.ts` (`GET /api/auth/me`).

4. Route map (as built)

Route	Purpose	Key files
<code>/</code>	Home: hero + featured gear	<code>app/(publicGroup)/page.tsx</code>
<code>/gears</code>	Search/filter/paginate listings	<code>app/(publicGroup)/gears/</code>
<code>/gears/[id]</code>	Gear detail + Rent Now panel	<code>app/(publicGroup)/gears/[id]/</code>
<code>/gears/[id]/reviews</code>	Public reviews (paginated)	<code>app/(publicGroup)/gears/[id]/reviews/</code>
<code>/about</code> <code>/contact</code> <code>/services</code>	Static pages	<code>app/(publicGroup)/...</code>
<code>/login</code> <code>/register</code>	Auth forms	<code>app/(authGroup)/...</code>
<code>/customer-dashboard</code>	Orders, payments, review	<code>app/(dashboardGroup)/customer-dashboard/**</code>
<code>/customer-dashboard/orders/[id]/pay</code>	Pay page	<code>app/(dashboardGroup)/customer-dashboard/orders/[id]/pay/</code>
<code>/provider-dashboard</code>	Overview, inventory, orders	<code>app/(dashboardGroup)/provider-dashboard/**</code>
<code>/provider-dashboard/gear/new</code> <code>/gear/[id]/edit</code>	Gear forms	<code>app/(dashboardGroup)/provider-dashboard/gear/**</code>
<code>/admin-dashboard</code>	Stats, users, gear, orders, categories	<code>app/(dashboardGroup)/admin-dashboard/**</code>
<code>/payment/success</code> <code>/payment/cancel</code>	SSLCommerz return pages	<code>app/(publicGroup)/payment/**</code>
<code>/profile</code>	Account (read-only view: avatar, name, email, phone, role, member since)	<code>app/(publicGroup)/profile</code>

`/settings` was removed — it redirects to `/profile` . The account page is a read-only info view; editing lives on the backend (`PATCH /api/auth/profile`), so no frontend settings form is shipped.

Middleware: `proxy.ts` . Root loading/error/not-found: `app/loading.tsx` , `app/error.tsx` , `app/not-found.tsx` .

5. Customer journey

1. **Browse & filter** (`/gears`): search term, category, brand, min/max price, availability date range — URL-driven, React Query caching (`lib/api/gear.ts` , filters in `GearFilters.tsx`).

2. **Gear detail** (`/gears/[id]`): image gallery, specs, provider info, `RentNowPanel.tsx` — pick dates + quantity; past/unavailable dates blocked.
3. **Place rental** (server action `rentalActions.ts`): validates dates (overlap with existing bookings), creates order (`POST /api/rentals`), status `PLACED` .
4. **Pay** (`/customer-dashboard/orders/[id]/pay`): `POST /api/payments/create` → SSLCommerz sandbox redirect. Return pages verify status before rendering.
5. **Track**: order history with status badges; cancel while `PLACED` .
6. **Review**: after gear returned, one review per order (`reviewActions.ts` , `POST /api/reviews`).

6. Provider dashboard

- Overview: total gear, active rentals, pending orders.
- Inventory (`my-gear` , `gear/new` , `gear/[id]/edit`): add/edit/delete gear, image upload via `CldUploadWidget` (`gear-image-upload.tsx`).
- Orders (`orders`): paginated list with status filter tabs; advance an order through `PLACED` → `CONFIRMED` → `PICKED_UP` → `RETURNED` (or cancel), with optimistic UI + toasts. `lib/api/provider.ts` .

7. Admin dashboard

- Stats overview: total users, active gear, total rentals.
- Users: search + pagination, suspend/activate (`PATCH /api/admin/users/:id`).
- Gear & orders: global moderation tables.
- Categories: add / edit / delete (delete blocked while category is in use). `lib/api/admin.ts` , `lib/api/categories.ts` .

8. Payment loop (SSLCommerz sandbox)

Customer pays → Backend creates SSLCommerz session (`BACKEND_PUBLIC_URL`)

- Customer pays in sandbox → SSLCommerz callback to backend
- Backend updates order/payment → redirects to `FRONTEND_URL /payment/success?orderId=...`
- Frontend verifies status (`GET /api/payments/status/:orderId`) and shows success page

Key files: `createPaymentAction.ts` , `getPaymentStatus.ts` , `app/(publicGroup)/payment/success|cancel` .

9. Error & loading states (deliberately visible)

- Skeleton `loading.tsx` on every server-fetching route.
- `error.tsx` boundaries with "Try again" / "Go home".
- `not-found.tsx` pages.
- `sonner` toasts on **every** mutation (success + failure).
- Zod schemas emit human-readable inline messages per field.

10. API surface

Full endpoint inventory (method / path / auth / source file) is in `API_INTEGRATION.md` . Client wrappers live in `lib/api/*.ts` ; server fetchers in `service/*` .

Part 2 — Live demo script (for the examiner)

Total ~7–10 minutes. Log in as each role before each section.

1. Roles tour (~1 min)

"One app, three dashboards. The navbar and middleware redirect based on the logged-in role."

- Show `/login` → log in as the **admin**, point at the admin dashboard.
- Open `/register` in a private window, show the role selector.

2. Customer journey (~2 min)

- Log in as a customer. Browse `/gears`.
- Apply a category + price filter; point at the URL (state is URL-driven).
- Open a gear detail page; select rental dates in the Rent Now panel.
- Place the order; show the success toast and the order in the dashboard.

3. Payment loop (~2 min)

- From the order, click **Pay Now** → SSLCommerz sandbox page.
- Complete a test payment (sandbox card). Land on `/payment/success`.
- Show order status updating in the customer dashboard.

4. Provider dashboard (~2 min)

- Log in as a provider. Add a gear item with a Cloudinary image upload.
- Manage an order: CONFIRM → PICKED UP → RETURNED. Point at optimistic UI + toasts.

5. Admin dashboard (~1 min)

- Log in as `admin@gearup.com`. Suspend a user, then re-activate.
- Add/rename a category in the Categories tab.

6. Error & loading states (~1–2 min)

- Deliberately trigger: submit the gear form with a bad price (Zod error), open `/gears` on a slow connection to show the skeleton, hit a bad URL to show `not-found`, trigger a failed mutation to show the error toast.

Close: "Live frontend and backend, admin creds in the README, `API_INTEGRATION.md` documents every endpoint."